

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Графы

Вариант № 9

Студент: **Чернышева А.П.**

Группа: **ИУ7-36Б**

Преподаватель: **Силантьева А. В.**

2025 г.

Описание условия задачи

Написать программу для работы с системой двусторонних дорог между городами, представленной в виде ориентированного взвешенного графа, где вес ребра соответствует расстоянию между городами. Двусторонние дороги могут иметь разную длину в разные стороны. Граф является связным (между любыми двумя городами существует путь). Требуется найти город (вершину), для которого сумма расстояний до всех остальных городов минимальна.

Описание ТЗ

Описание исходных данных

Количество городов (вершин графа) и количество дорог (рёбер) – натуральные числа.

Для каждой дороги: номера двух городов и расстояние между ними – натуральные числа.

Имя файла(строка не превышающая 255 символов).

Ограничение: максимальное количество городов — 100.

Введенный пункт меню.

Результаты:

Информация о графе (количество вершин, рёбер, тип, связность, использование памяти).

Найденный город с минимальной суммой расстояний до остальных.

Суммарное расстояние от этого города до всех остальных.

Матрица кратчайших расстояний между всеми парами городов (по алгоритму Дейкстры).

Визуализация графа в формате DOT с выделением найденного оптимального города и кратчайших путей от него.

Время выполнения вычислений и объём используемой памяти.

Описание задачи, реализуемой программой

Программа реализует систему работы с графами:

1. Загрузить граф из файла.
2. Ввести граф вручную.
3. Найти город с минимальной суммой расстояний.
4. Экспортировать граф в DOT-формат.
5. Показать информацию о графе.

Способ обращения к программе

Программа запускается через исполняемый файл app.exe.

Описание возможных аварийных ситуаций и ошибок пользователя

1. Ошибка открытия файла (неверное имя или отсутствие файла).
2. Неверный формат данных в файле (не числа, неверное количество чисел).
3. Ввод некорректных значений (отрицательные расстояния, номера городов вне диапазона, петли).
4. Превышение максимального количества вершин (более 100).
5. Несвязный граф (по условию граф должен быть связным, программа предупреждает, но продолжает работу).

6. Ошибки выделения памяти
7. Невалидный ввод пункта меню.

Описание внутренних структур данных

Edge – структура ребра графа

```
typedef struct Edge {
    int start;
    int finish;
    int cost;
    struct Edge *next;
} Edge;
```

start, finish — номера вершин, соединённых ориентированным ребром.
cost — длина ребра (положительное целое число).
next — указатель на следующее ребро в односвязном списке рёбер для данной вершины

Node – структура вершины графа(узел)

```
typedef struct Node {
    int id;
    Edge *edges;
} Node;
```

id — уникальный номер вершины (от 0 до size-1).
edges — указатель на голову односвязного списка исходящих рёбер.

Graph – структура графа

```
typedef struct Graph {
    size_t size;
    Node **nodes;
} Graph;
```

size — количество вершин в графе.
nodes — динамический массив указателей на вершины.
MAX_VERTICES = 100 — максимальное количество вершин.
INF = 1000000 — значение, обозначающее бесконечность (отсутствие пути).
dist[MAX_VERTICES][MAX_VERTICES] — матрица кратчайших расстояний.

Общий алгоритм программы:

1. **Инициализация программы:**
   - Объявление указателя на граф
   - Инициализация переменных для управления состоянием программы
2. **Основной цикл программы:**
   - Вывод меню с доступными операциями
   - Ожидание ввода выбора пользователя
   - Выполнение выбранной операции
3. **Операции меню:**

1. Загрузить граф из файла.
2. Ввести граф вручную.
3. Найти город с минимальной суммой расстояний.

4. Экспортировать граф в DOT-формат.
5. Показать информацию о графе.
6. Выход.

Описание алгоритмов функций

graph_read(FILE *f, Graph **graph) – чтение ориентированного графа из файла или стандартного ввода.

1. Считывание количества вершин и рёбер.
2. Проверка корректности данных (диапазоны, положительные веса).
3. Создание ориентированного графа с заданным количеством вершин.
4. Для каждого ребра:
   - Считывание номеров вершин (отправления и назначения) и веса.
   - Проверка на петли и корректность номеров.
   - Создание одного ориентированного ребра и добавление в список исходящих рёбер начальной вершины.
5. В случае ошибки — освобождение ресурсов и возврат кода ошибки.

dijkstra(Graph *graph, int start, int dist[]*)* – вычисление кратчайших расстояний от вершины start до всех остальных вершин.

1. Инициализация массива dist[] значением INF, dist[start] = 0.
2. Массив visited[] для отслеживания посещённых вершин.
3. Для каждой вершины:
   - Выбор непосещённой вершины с минимальным значением dist[].
   - Помечание вершины как посещённой.
   - Обновление расстояний до соседей через выбранную вершину.
4. Возврат массива dist[].

all_pairs_dijkstra(Graph *graph, int dist[][MAX_VERTICES*])*— вычисление кратчайших путей между всеми парами вершин.

1. Для каждой вершины i:
   - Вызов dijkstra(graph, i, dist[i]).
2. Заполнение матрицы расстояний.

find_city_min_total_distance(Graph *graph, int *city, int *total_distance)— поиск города с минимальной суммой расстояний до всех остальных.

1. Проверка связности графа (предупреждение, если граф несвязный).
2. Вызов all_pairs_dijkstra для получения матрицы расстояний.
3. Для каждой вершины i:
   - Вычисление суммы расстояний от i до всех остальных вершин.
   - Если все расстояния конечны (вершина достижима) и сумма меньше текущего минимума, обновить минимум и запомнить вершину.
4. Если минимум не найден (граф несвязный), возврат ошибки.
5. Запись результата в *city и *total_distance.

graph_print_min_city_dot(FILE *f, Graph *graph, int min_city) – генерация DOT-описания ориентированного графа с выделением оптимального города и кратчайших путей от него.

1. Запись заголовка ориентированного графа.

2. Вычисление расстояний от оптимального города до всех остальных через dijkstra.
3. Для каждой вершины:
    - Если вершина — оптимальный город, выделение её цветом и подписью.
    - Иначе — подпись с расстоянием от оптимального города.
4. Для каждого ребра:
    - Определение, входит ли ребро в кратчайший путь от оптимального города к какой-либо вершине (проверка условия dist[u] + weight == dist[v]).
    - Если входит — рисование ребра красным цветом с увеличенной толщиной.
    - Иначе — чёрным цветом.
5. Запись завершения графа.

graph_is_connected(Graph *graph) – проверка ориентированного графа на связность (сильная связность).

1. Для каждой вершины start:
    - Запуск DFS из вершины start.
    - Проверка, что все вершины достижимы из start.
2. Если для всех стартовых вершин условие выполняется, граф сильно связен.
3. В противном случае — граф не является сильно связным.

edge_create(int start, int finish, int cost) – создание нового ребра ориентированного графа.

1. Выделение памяти размером sizeof(Edge) с помощью malloc().
2. Проверка успешности выделения памяти:
    - Если память не выделена, возврат NULL.
3. Инициализация полей структуры
4. Возврат указателя на созданное ребро.

node_create(int id) – создание новой вершины графа.

1. Выделение памяти размером sizeof(Node) с помощью malloc().
2. Проверка успешности выделения памяти:
    - Если память не выделена, возврат NULL.
3. Инициализация полей структуры
4. Возврат указателя на созданную вершину

graph_create(size_t size, bool directed) – создание графа с заданным количеством вершин.

1. Выделение памяти для структуры Graph:

text

Graph *graph = malloc(sizeof(Graph))

Проверка на успешность выделения.

2. Инициализация полей графа
3. Выделение памяти для массива указателей на вершины:
4. Построение массива вершин

5. Обработка ошибок при создании вершин
6. Возврат указателя на созданный граф.Освобождение памяти

edge_destroy(Edge *edge) – освобождение памяти, занятой ребром.

1. Проверка, что указатель edge не NULL.
2. Вызов free(edge) для освобождения памяти.

node_destroy(Node *node) – рекурсивное освобождение памяти вершины и всех её исходящих рёбер.

1. Получение указателя на первый элемент списка рёбер:
2. Последовательный обход списка рёбер
3. Освобождение памяти самой вершины

graph_destroy(Graph *graph) – полное освобождение памяти, занятой графом.

1. Проверка, что указатель graph не NULL.
2. Последовательное уничтожение всех вершин графа
3. Освобождение памяти массива указателей на вершины
4. Освобождение памяти структуры графа

graph_count_edges(Graph *graph) – подсчёт общего количества рёбер в графе.

1. Инициализация счётчика: count = 0
2. Для каждой вершины i от 0 до size-1 подсчет количества ребер
3. Возврат значения count

graph_memory_usage(Graph *graph) – оценка объёма памяти, используемого графом.

1. Инициализация счётчика:
2. Для каждой вершины:
    - Добавление размера структуры Node
    - Обход списка рёбер вершины
3. Возврат общего объёма памяти в байтах.

graph_has_edge(Graph *graph, int from, int to, int *cost) – проверка существования ребра из вершины from в вершину to.

1. Проверка корректности параметров
2. Получение списка рёбер, исходящих из вершины
3. Линейный поиск в списке рёбер
4. Если ребро не найдено - возврат false.

print_shortest_paths_matrix(Graph *graph) – вывод матрицы кратчайших расстояний между всеми парами вершин.

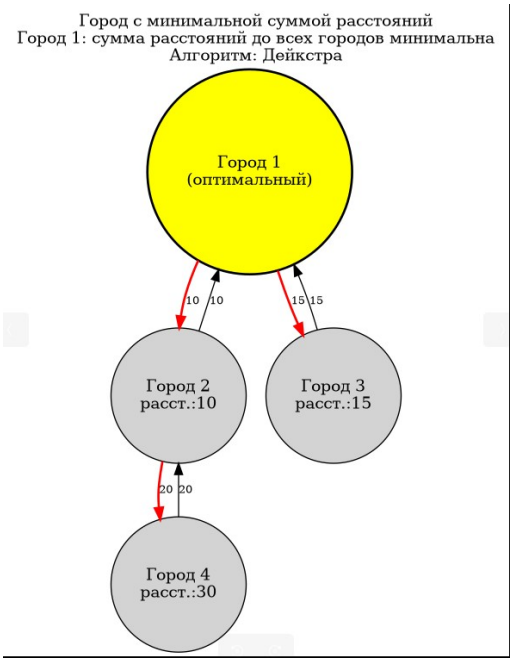
1. Вычисление матрицы расстояний через all_pairs_dijkstra(graph, dist).
2. Форматированный вывод заголовка матрицы.
3. Построение шапки таблицы с номерами вершин
4. Вывод строк матрицы

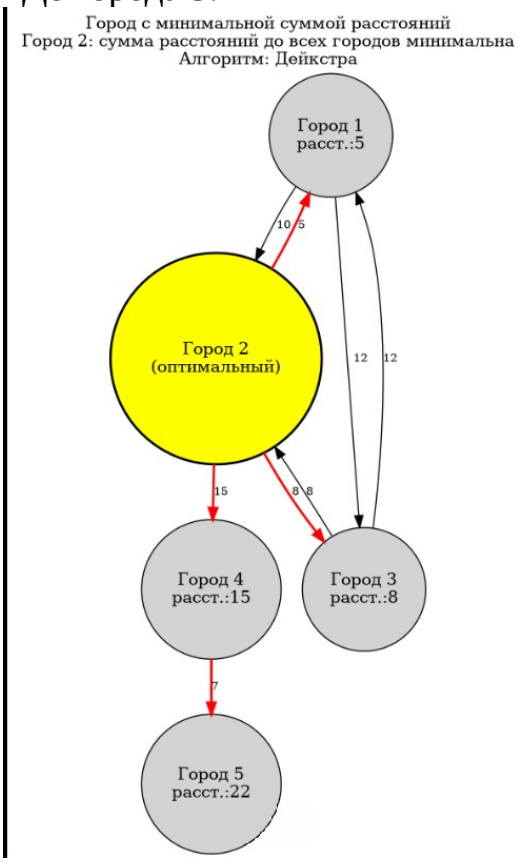
5. Вывод суммы расстояний для каждой вершины:

Тесты

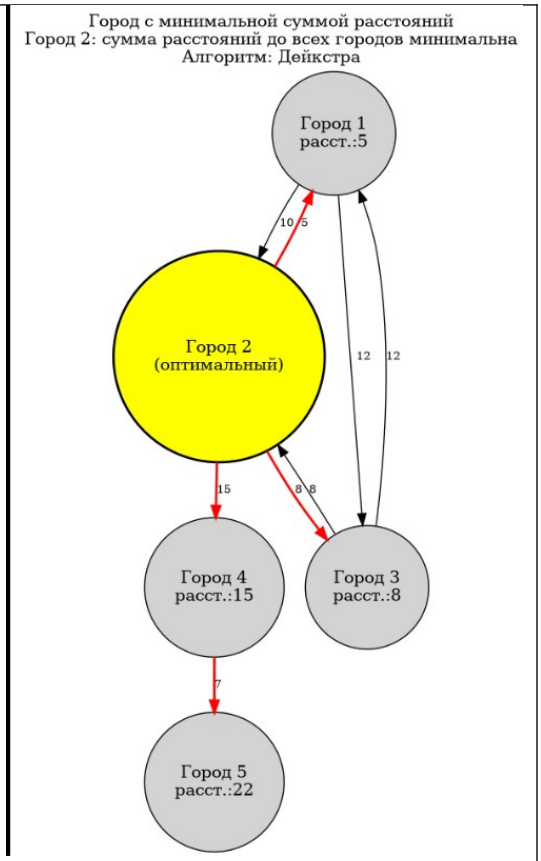
Позитивные тесты

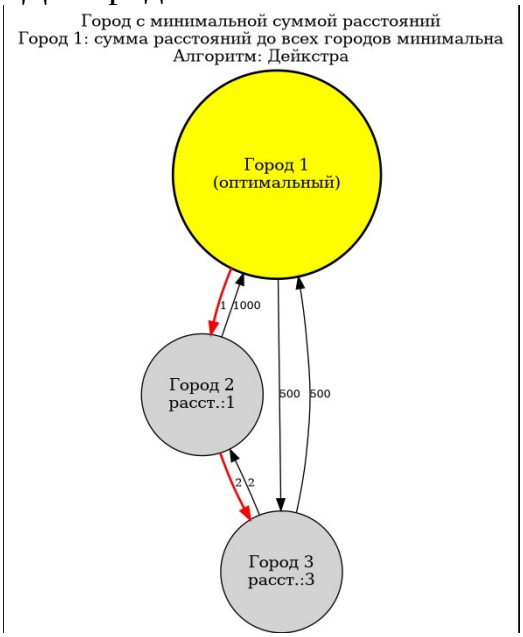
Номер теста	Описание	Ожидаемый результат
1	Граф с симметричными дорогами 4 6 1 2 10 2 1 10 1 3 15 3 1 15 2 4 20 4 2 20	МАТРИЦА КРАТЧАЙШИХ РАССТОЯНИЙ 1 2 3 4 ---------- 1 | 0 10 15 30 2 | 10 0 25 20 3 | 15 25 0 45 4 | 30 20 45 0 СУММА РАССТОЯНИЙ ДО ДРУГИХ ГОРОДОВ Город 1: сумма = 55, среднее = 18.33 Город 2: сумма = 55, среднее = 18.33 Город 3: сумма = 85, среднее = 28.33 Город 4: сумма = 95, среднее = 31.67 РЕЗУЛЬТАТ Оптимальный город: 1 Сумма расстояний до всех остальных городов: 55 Среднее расстояние до других городов: 18.33 Расстояния от города 1 до других городов: До города 2: 10 До города 3: 15 До города 4: 30

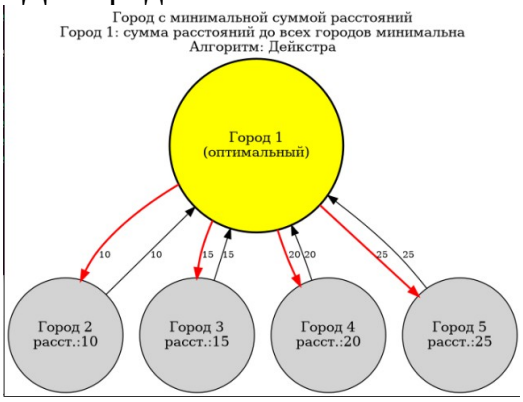
		
2	Граф с разными длинами в разные стороны 5 8 1 2 10 2 1 5 1 3 12 3 1 12 2 3 8 3 2 8 2 4 15 4 5 7	МАТРИЦА КРАТЧАЙШИХ РАССТОЯНИЙ (алгоритм Дейкстры) 1 2 3 4 5 --- 1 | 0 10 12 25 32 2 | 5 0 8 15 22 3 | 12 8 0 23 30 4 | ∞ ∞ ∞ 0 7 5 | ∞ ∞ ∞ ∞ 0 СУММА РАССТОЯНИЙ ДО ДРУГИХ ГОРОДОВ Город 1: сумма = 79, среднее = 19.75 Город 2: сумма = 50, среднее = 12.50 Город 3: сумма = 73, среднее = 18.25 Город 4: НЕСВЯЗАН со всеми городами Город 5: НЕСВЯЗАН со всеми городами Внимание: граф не является связным! По условию задачи граф

		должен быть связным. РЕЗУЛЬТАТ Оптимальный город: 2 Сумма расстояний до всех остальных городов: 50 Среднее расстояние до других городов: 12.50 Расстояния от города 2 до других городов: До города 1: 5 До города 3: 8 До города 4: 15 До города 5: 22 
3	Несвязный граф 6 5 1 2 10 2 1 10 3 4 8 4 3 8 5 6 12	МАТРИЦА КРАТЧАЙШИХ РАССТОЯНИЙ (алгоритм Дейкстры)

		1 2 3 4 5 6 ---------- 1 | 0 10 ∞ ∞ ∞ ∞ 2 | 10 0 ∞ ∞ ∞ ∞ 3 | ∞ ∞ 0 8 ∞ ∞ 4 | ∞ ∞ 8 0 ∞ ∞ 5 | ∞ ∞ ∞ ∞ 0 12 6 | ∞ ∞ ∞ ∞ ∞ 0 СУММА РАССТОЯНИЙ ДО ДРУГИХ ГОРОДОВ Город 1: НЕСВЯЗАН со всеми городами Город 2: НЕСВЯЗАН со всеми городами Город 3: НЕСВЯЗАН со всеми городами Город 4: НЕСВЯЗАН со всеми городами Город 5: НЕСВЯЗАН со всеми городами Город 6: НЕСВЯЗАН со всеми городами Внимание: граф не является связным! По условию задачи граф должен быть связным. Не удалось найти город с минимальной суммой расстояний.

		
4	Граф с большими значениями дорог 3 6 1 2 1 2 1 1000 1 3 500 3 1 500 2 3 2 3 2 2	МАТРИЦА КРАТЧАЙШИХ РАССТОЯНИЙ (алгоритм Дейкстры) 1 2 3 --- 1 | 0 1 3 2 | 502 0 2 3 | 500 2 0 СУММА РАССТОЯНИЙ ДО ДРУГИХ ГОРОДОВ Город 1: сумма = 4, среднее = 2.00 Город 2: сумма = 504, среднее = 252.00 Город 3: сумма = 502, среднее = 251.00 РЕЗУЛЬТАТ Оптимальный город: 1 Сумма расстояний до всех остальных городов: 4

		Среднее расстояние до других городов: 2.00 Расстояния от города 1 до других городов: До города 2: 1 До города 3: 3 
5	Граф с периферией 5 8 1 2 10 2 1 10 1 3 15 3 1 15 1 4 20 4 1 20 1 5 25 5 1 25	МАТРИЦА КРАТЧАЙШИХ РАССТОЯНИЙ (алгоритм Дейкстры) 1 2 3 4 5 --- 1 | 0 10 15 20 25 2 | 10 0 25 30 35 3 | 15 25 0 35 40 4 | 20 30 35 0 45 5 | 25 35 40 45 0 СУММА РАССТОЯНИЙ ДО ДРУГИХ ГОРОДОВ Город 1: сумма = 70, среднее = 17.50 Город 2: сумма = 100, среднее = 25.00 Город 3: сумма = 115, среднее = 28.75 Город 4: сумма = 130, среднее =

		32.50 Город 5: сумма = 145, среднее = 36.25 РЕЗУЛЬТАТ Оптимальный город: 1 Сумма расстояний до всех остальных городов: 70 Среднее расстояние до других городов: 17.50 Расстояния от города 1 до других городов: До города 2: 10 До города 3: 15 До города 4: 20 До города 5: 25 

Негативные тесты

Номер теста	Описание	Ожидаемый результат
1	Загрузка несуществующего файла 4	Ошибка открытия файла 4
2	Файл с некорректным форматом (буквы вместо чисел)	Ошибка чтения графа
3	Операции с незагруженными данными 5 7 a s d	
4	Ввод невалидного номера меню	Неверный выбор
5	Ввод отрицательного расстояния	Ошибка: расстояние должно

	Ребро 1 (ориентированное): 1 2 -10	быть > 0 Ошибка ввода графа
6	Ввод номера города вне диапазона Ребро 1 (ориентированное): 1 122 3	Ошибка: номера городов должны быть от 1 до 5 Ошибка ввода графа
7	Граф с петлёй (город соединён сам с собой) Ребро 1 (ориентированное): 1 1 43	Ошибка: петли не допускаются (город не может быть связан сам с собой) Ошибка ввода графа
8	Пустой ввод (нулевое количество вершин) Введите количество городов и дорог: Введите количество вершин и ребер: 0 0	Количество вершин должно быть > 0 Ошибка ввода графа

Вывод

Алгоритм Дейкстры был выбран по следующим ключевым причинам:

1.Алгоритм Дейкстры необходим чтобы находить расстояния от каждого города до всех остальных, что в этой задаче и происходить. Сложность $O(V^2 \log V)$

2.Работа с неотрицательными весами рёбер.

3.Эффективность для разреженных графов.

Почему список смежности:

1. Экономия памяти для разреженных графов.

2. Эффективный обход соседей вершины - Для алгоритма Дейкстры критически важен быстрый доступ ко всем рёбрам, исходящим из текущей вершины.

Список смежности предоставляет прямой доступ к этому списку за O(1).

Комбинация алгоритма Дейкстры и представления графа в виде списка смежности образует оптимальное решение для поставленной задачи, обеспечивая правильность вычислений, эффективное использование ресурсов

Контрольные вопросы

1. Что такое граф?

Граф - это абстрактная математическая структура, представляющая собой множество вершин и рёбер, соединяющих их. Графы применяются для моделирования отношений и связей между объектами.

2. Как представляются графы в памяти?

Графы могут быть представлены в памяти различными способами. Два основных метода - это матрица смежности и список смежности. В матрице смежности графа, элемент (i, j) равен 1, если между вершинами i и j есть ребро,

и 0 в противном случае. В списке смежности для каждой вершины хранится список её соседей.

3. Какие операции возможны над графами?

- Добавление и удаление вершин и рёбер.
- Поиск пути между вершинами.
- Проверка наличия ребра между вершинами.
- Обход графа в глубину или в ширину.
- Нахождение кратчайшего пути между вершинами.
- Проверка связности графа.
- Поиск циклов в графе.

4. Какие способы обхода графов существуют?

Обход в глубину (DFS): Посещение вершин вглубь, до тех пор, пока не достигнута конечная вершина, затем возврат и продолжение к следующей.

Обход в ширину (BFS): Посещение ближайших вершин перед переходом к вершинам, находящимся на бóльшем расстоянии.

5. Где используются графовые структуры?

Социальные сети: Представление связей между пользователями.

Транспортные сети: Моделирование дорог, путей следования и транспортных узлов.

Биоинформатика: Анализ генетических связей и взаимодействий.

Графические системы: Определение связей между объектами в компьютерной графике.

Роутинг в компьютерных сетях: Поиск кратчайших путей между узлами сети.

6. Какие пути в графе Вы знаете?

Простой путь: Путь, в котором все вершины различны.

Цикл: Замкнутый путь, который начинается и заканчивается в одной и той же вершине, и содержит хотя бы одну реберную дугу.

Простой цикл: Цикл, в котором все вершины, кроме начальной и конечной, различны.

Путь с повторяющимися вершинами: Путь, в котором некоторые вершины могут повторяться.

Эйлеров путь в графе — это путь, проходящий по всем рёбрам графа и притом только по одному разу.

Гамильтоновым циклом является такой цикл, который проходит через каждую вершину данного графа ровно по одному разу; то есть простой цикл, в который входят все вершины графа.

7. Что такое каркасы графа?

Каркас графа - это связный подграф, содержащий все вершины исходного графа и являющийся деревом (не содержит циклов). Каркас обладает минимальным возможным весом среди всех подобных связных подграфов. Каркасы часто используются в задачах построения эффективных сетей связ